



Project Title: Exam Security
System Identity Verification
Seating Plan Violation
Logging

Prepared by:

Berkay Aksoy

Number: 220706307

Hakkı Dökmeci

Number:2207060026

**Course: SE 342 01 Software Validation and
Testing Instructor: Emre Olca**

UNIT TESTING

TESTING & VALIDATION

Unit Testing Strategy: CheckInService

This section documents the unit testing framework implemented for the CheckInService component. The primary objective of these tests is to verify that critical business rules regarding student attendance and security are rigorously enforced at the backend level, ensuring logic integrity independent of the user interface.

The tests are implemented using the **MSTest** framework and executed against an **in-memory Entity Framework Core** database. This strategy ensures isolation, repeatability, and independence from external systems such as the production database or the live Machine Learning service.

Test Environment and Setup

To guarantee consistent execution, each test method utilizes a standardized setup strategy initiated via the [TestInitialize] method:

- **Isolated Database:** A fresh AppDbContext instance is created using EF Core's *InMemory* provider for every test run. This prevents data contamination between tests.
- **Data Seeding:** Minimal required test data (e.g., Users, Students, Exams) is injected using a TestSeed.SeedMinimal method to establish a baseline state.
- **Service Mocking:** A stub implementation of the face verification service (StubFaceMatchService) is injected. This allows the test suite to deterministically control identity verification outcomes (e.g., forcing a "Match" or "No Match" result) without making external API calls.

Test Case Definitions

The following test scenarios cover the critical paths and edge cases of the check-in workflow.

1. Test Case: Valid Check-in (Happy Path)

- **Method Name:**
CreateCheckIn_Valid_Should_Save_CheckedIn_And_No_Violation()
- **Purpose:** Verifies the standard workflow where all security conditions are met.
- **Scenario:**
 - The student is enrolled in the exam.

- The observed seat matches the assigned seat.
- Identity verification returns a positive result.
- **Expected Behavior:**
 - A new record is saved with the status CHECKED_IN.
 - Identity result is recorded as MATCH and Seat result as OK.
 - **No** violation records are generated in the system.
- **Significance:** Confirms that compliant students are processed correctly without false positives.

2. Test Case: Duplicate Check-in Prevention

- **Method Name:**
CreateCheckIn_Duplicate_Should_Throw_And_Not_Add_Second_Record()
- **Purpose:** Enforces the business rule that a student cannot verify attendance multiple times for the same session.
- **Scenario:** A student who has already successfully checked in attempts a second check-in for the same exam.
- **Expected Behavior:**
 - The system rejects the second request.
 - A specific exception is thrown to indicate the rule violation.
 - The database integrity is preserved (no duplicate records are added).
- **Significance:** Critical for exam integrity; ensures that the rule is enforced by the backend logic and cannot be bypassed by frontend manipulation.

3. Test Case: Seating Compliance Validation

- **Method Name:**
CreateCheckIn_WrongSeat_Should_Save_Violation_And_Log_WRONG_SEAT()
- **Purpose:** Verifies that the system correctly identifies and logs seating anomalies.
- **Scenario:**
 - Identity verification succeeds.

- The observed seat code entered by the proctor does *not* match the student's assigned seat.
- **Expected Behavior:**
 - The check-in status is recorded as VIOLATION.
 - The Seat result is marked as NOT_OK.
 - A corresponding entry is created in the Violations table with the type WRONG_SEAT.
- **Significance:** Ensures that seating enforcement is automated and persistent.

4. Test Case: Identity Verification Enforcement

- **Method Name:**
CreateCheckIn_IdentityMismatch_Should_Save_Violation_And_Log_IDENTITY_MISMATCH()
- **Purpose:** Validates the system's response to biometric verification failures.
- **Scenario:**
 - The student is in the correct seat.
 - The face verification service returns a mismatch (low confidence score).
- **Expected Behavior:**
 - The check-in status is recorded as VIOLATION.
 - Identity result is marked as NO_MATCH.
 - A corresponding entry is created in the Violations table with the type IDENTITY_MISMATCH.
- **Significance:** Confirms that the core security feature (Face ID) is functional and that failures are logged for auditing.

Conclusion

These unit tests collectively demonstrate that the CheckInService is robust, modular, and secure. By validating these scenarios in an isolated environment, we confirm that:

1. **Backend Authority:** All critical security rules are enforced by the server, not the client.

2. **Automated Detection:** Violations (Wrong Seat, Identity Mismatch) are automatically detected and persisted.
3. **Data Integrity:** Invalid or duplicate operations are prevented at the database level.

Harika, bu metni de bir önceki bölümle uyumlu olacak şekilde **akademik, yapılandırılmış ve profesyonel** bir formata dönüştürdüm.

Bu metni raporunun "**6.4. Test Case Classification & Coverage**" başlığı altına doğrudan yapıştırabilirsin.

TEST CASE CLASSIFICATION & COVERAGE ANALYSIS

The implemented unit testing suite is structured to provide comprehensive coverage through three distinct categories: **Functional**, **Negative**, and **Edge Case** scenarios. This classification ensures that the system is validated not only for expected behaviors but also for its resilience against invalid inputs and boundary conditions. Each test case includes explicitly defined inputs and expected assertions to ensure unambiguous validation of the backend logic.

6 Functional Test Cases (Positive Testing)

Functional tests validate that the system operates correctly under normal and expected conditions, ensuring the "Happy Path" workflow is robust.

- **Test Case: Valid Check-In Workflow**
 - **Method:**
CreateCheckIn_Valid_Should_Save_CheckedIn_And_No_Violation
 - **Scenario:** A registered student attempts to check in while sitting in the correct assigned seat, and the identity verification service returns a positive match.
 - **Expected Result:**
 - A new record is committed to the database.
 - **Status:** CHECKED_IN
 - **Identity Result:** MATCH
 - **Seat Result:** OK
 - **Violations:** No violation records are generated.
 - **Validation:** This test confirms that the backend correctly processes and persists valid check-in operations without generating false alarms.

Negative Test Cases (Error Handling)

Negative tests verify that the system enforces business rules by correctly rejecting invalid operations and preserving data integrity.

- **Test Case: Duplicate Check-In Prevention**

- **Method:**
CreateCheckIn_Duplicate_Should_Throw_And_Not_Add_Second_Record
- **Scenario:** A student who has already successfully checked in attempts to initiate a second check-in transaction for the same exam session.
- **Expected Result:**
 - The system rejects the second request immediately.
 - An exception or conflict response is triggered.
 - **Database State:** No additional record is added; the integrity of the unique check-in constraint is preserved.
- **Validation:** This ensures the strict enforcement of the rule: *A student cannot check in more than once per exam.*

Edge Case Scenarios (Boundary Testing)

Edge cases validate the system's behavior under specific exceptional conditions where one factor is valid while another is invalid. These tests are critical for the "Violation Logging" logic.

- **Edge Case A: Wrong Seat with Valid Identity**

- **Method:**
CreateCheckIn_WrongSeat_Should_Save_Violation_And_Log_WRONG_SEAT
- **Scenario:** The identity verification is successful (face matches), but the student is sitting in a seat different from their assigned one.
- **Expected Result:**
 - **Status:** VIOLATION
 - **Seat Result:** NOT_OK
 - **Log:** A violation record with type WRONG_SEAT is automatically created.
- **Validation:** This validates that seating rules are enforced independently of identity verification results.

- **Edge Case B: Identity Mismatch with Correct Seat**

- **Method:**
CreateCheckIn_IdentityMismatch_Should_Save_Violation_And_Log_IDENTITY_MISMATCH
- **Scenario:** The student is sitting in the correct assigned seat, but the biometric verification fails (low confidence score).

- **Expected Result:**
 - **Status:** VIOLATION
 - **Identity Result:** NO_MATCH
 - **Log:** A violation record with type IDENTITY_MISMATCH is automatically created.
- **Validation:** This confirms that identity failures are detected and logged even when the seating compliance is satisfied.

Verification Criteria & Determinism

To ensure that test outcomes are measurable and verifiable, each test case implements the following strict assertions:

1. **State Assertion:** The expected check-in status (CheckedIn vs. Violation) is explicitly asserted.
2. **Logic Branching:** The specific reasons for success or failure (Identity vs. Seat) are verified.
3. **Side Effects:** The presence or absence of related entries in the Violations table is checked.
4. **Persistence:** Database state changes (record counts) are validated to ensure no phantom reads or writes occur.